Huellitas — Red de Clínicas y Derivación de Laboratorio

TP Integrador 2026 · Desarrollo de Aplicaciones con Objetos · Tema 2 · Versión de entrega, 1 de noviembre de 2026

1. Introducción

Huellitas integra la atención clínica de una red de tres clínicas veterinarias con la derivación de muestras a un laboratorio central, y registra cada paso de la cadena de custodia desde la extracción hasta el diagnóstico.

El núcleo del modelo son dos ciclos de vida: el de la **muestra biológica** (Extraída → Empaquetada → En tránsito → Recibida → Analizada, o Muestra Rechazada) y el del **remito biológico** que la acompaña (Emitido → En tránsito → Recibido, Rechazado o Anulado). Ambos se modelan con el patrón State y cada cambio queda en una bitácora inmutable.

Aspecto	Decisión
Lenguaje	Python 3.11 o superior
Interfaz	Web local: Flask + plantillas Jinja + Bootstrap 5 (gráficos con Chart.js)
Persistencia	SQLite, acceso con SQL propio (sin ORM)
Dependencias externas	Solo Flask; Bootstrap y Chart.js van incluidos en el proyecto (funciona sin internet)
Alcance	7 procesos obligatorios, 1 opcional parcial, 5 patrones (State aplicado a la muestra y al remito), 6 reportes
Pruebas	42 pruebas automatizadas: 34 de negocio y 8 de la interfaz web

2. Casos de uso nuevo

Cinco actores intervienen en el sistema. El administrativo y el veterinario trabajan en la clínica; logística de la clínica arma y despacha los envíos; el transportista registra los controles en ruta; el bioquímico recibe, verifica y analiza en el laboratorio central.

Cinco actores recorren el circuito de la muestra de punta a punta

Diagrama de casos de uso. Línea punteada = «include» / «extend».

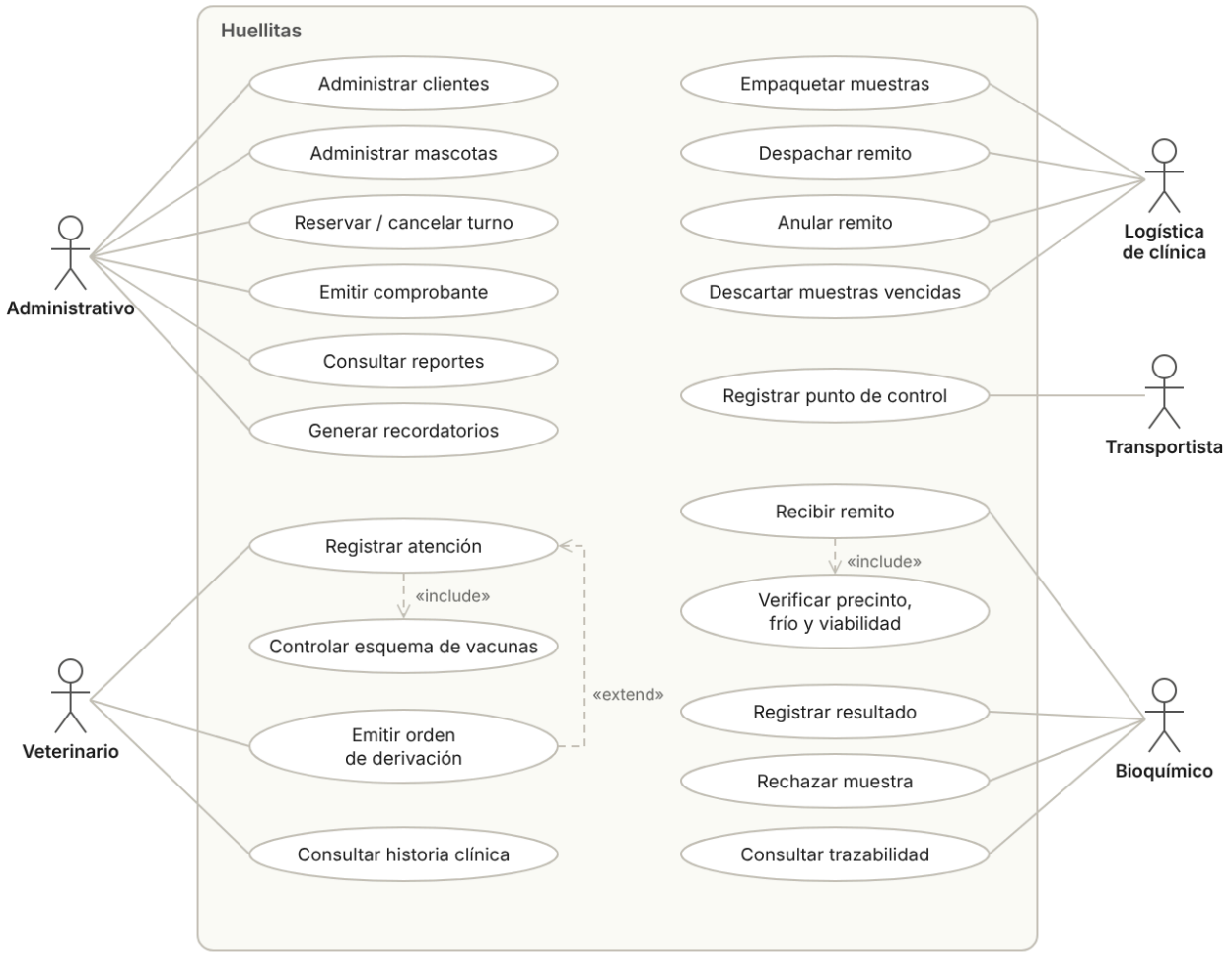


Diagrama de casos de uso

Actor	Responsabilidad	Pantallas
Administrativo	Alta y mantenimiento de clientes y mascotas, agenda de turnos, facturación, reportes y avisos	Clientes, Mascotas, Agenda, Comprobantes, Reportes, Avisos
Veterinario	Atiende, aplica vacunas, deriva análisis y consulta la historia clínica	Atención clínica, Historia clínica
Logística de clínica	Empaqueta muestras en conservadoras, emite remito y precinto, despacha o anula	Empaquetado, Remitos y envíos
Transportista	Informa los puntos de control con temperatura	Detalle del remito
Bioquímico	Recibe el remito, verifica precinto y cadena de frío, carga resultados o rechaza	Detalle del remito, Resultados, Trazabilidad

Especificación de los casos de uso principales

Caso de uso	Precondición	Flujo principal	Flujos alternativos
CU-01 Registrar atención y derivar	Mascota activa con responsable; veterinario activo	1. Elige paciente (el sistema muestra el esquema de vacunas). 2. Carga tipo, motivo, diagnóstico, tratamiento y vacunas. 3. El sistema valida el esquema «include» y registra. 4. «extend»: si deriva, elige análisis y prioridad; se crea una muestra Extraída por análisis.	Vacuna de otra especie, antes de la edad mínima o refuerzo prematuro: se rechaza. Fecha futura o anterior al nacimiento: se rechaza.
CU-02 Empaquetar muestras	Muestras Extraídas de la clínica; conservadora Disponible	1. Elige muestras y conservadora. 2. El sistema valida rango térmico, capacidad y viabilidad. 3. Emite remito (Emitido) y precinto; muestras → Empaquetada; conservadora → En uso.	Conservadora no apta, sin capacidad, muestras repetidas o vencidas: se rechaza sin cambios (transacción).
CU-03 Despachar y controlar envío	Remito Emitido	1. Carga transportista, vehículo y temperatura de salida. 2. Remito y muestras → En tránsito. 3. El transportista registra puntos de control con temperatura.	Anular antes de salir: precinto roto, muestras rechazadas, remito Anulado, conservadora liberada.
CU-04 Recibir remito	Remito En tránsito	1. Indica estado del precinto y temperatura de llegada. 2. «include» Verificar: por muestra, precinto → temperaturas de toda la cadena → horas de viabilidad. 3. Muestras viables → Recibida; remito → Recibido.	Precinto roto: todas las muestras → Muestra Rechazada y remito Rechazado. Falla de frío o viabilidad: se rechaza esa muestra.
CU-05 Registrar resultado	Muestra Recibida	1. Carga profesional, valores y conclusión. 2. Muestra → Analizada; se genera aviso al cliente.	Muestra Rechazada: el estado lo impide. Muestra hemolizada: el bioquímico la rechaza.
CU-06 Emitir comprobante	Atención sin comprobante; ninguna muestra pendiente	1. El sistema arma ítems: consulta, vacunas y análisis realizados. 2. Calcula total y numera.	Muestras sin resultado: no se emite. Las rechazadas no se facturan.

3. Modelo de dominio

El dominio tiene 24 entidades agrupadas en cinco áreas; cada una valida sus invariantes al construirse (`dominio/entidades.py`, `dominio/muestra.py`, `dominio/remito.py`), así un objeto inconsistente nunca llega a la base.

Área	Entidad	Atributos principales	Relación clave
Red	Clínica	nombre, dirección, teléfono	tiene veterinarios y conservadoras
Red	Laboratorio	nombre, dirección	destino de todos los remitos
Red	Veterinario	matrícula, especialidad, activo	pertenece a una clínica
Clientes	Cliente	DNI (único), nombre, contacto, activo	responsable de 1..n mascotas
Clientes	Mascota	especie, raza, sexo, nacimiento, peso	siempre tiene un cliente
Atención	Turno	inicio, fin, motivo, estado	veterinario + mascota
Atención	Consulta	fecha, tipo, motivo, diagnóstico, tratamiento, precio	forma la historia clínica
Atención	TipoVacuna / VacunaAplicada	especie, edad mínima, refuerzo (meses), lote	aplicada en una consulta
Atención	Comprobante / Ítem	número, total; tipo de servicio, subtotal	uno por consulta
Laboratorio	TipoAnálisis	tipo de muestra, rango °C, horas de viabilidad, precio	define cómo conservar la muestra
Laboratorio	OrdenDerivación	fecha, prioridad, observaciones	emitida desde una consulta
Laboratorio	Muestra	código, estado, motivo de rechazo	una por análisis de la orden
Laboratorio	HistorialEstado	estado anterior/nuevo, fecha, detalle	bitácora inmutable de la muestra
Custodia	Contenedor (conservadora)	código, rango °C, capacidad, estado	reutilizable, de una clínica
Custodia	Remito biológico	número, responsable, estado (State)	agrupa muestras en un contenedor
Custodia	HistorialEstadoRemito nuevo	estado anterior/nuevo, fecha, detalle	bitácora inmutable del remito
Custodia	Precinto sanitario	código, estado, fecha de verificación	uno por remito
Custodia	Envío / EventoEnvío	transportista; ubicación, temperatura, hora	eslabones de la cadena
Custodia	Recepción	recibido por, estado del precinto, °C llegada	cierra el remito

Custodia	Resultado	profesional, valores, conclusión	uno por muestra analizada
Avisos	Notificación	tipo, destinatario, mensaje, enviada	al cliente responsable

Regla de diseño importante: la muestra conoce su remito, y el remito conoce su contenedor y su precinto. Así, desde el código de una muestra se reconstruye toda su cadena de custodia.

Diagrama de clases. Reparte las entidades en dos partes y agrega dos diagramas con las clases de los patrones State y Observer: uno para la muestra y otro para el remito. Muestra los atributos y métodos principales; en el código, las asociaciones se guardan como ids.

Una mascota siempre tiene responsable y su atención forma la historia clínica

Clases 1 de 4: red, clientes, atención y facturación. Rombo = composición. Todas validan sus datos al construirse.

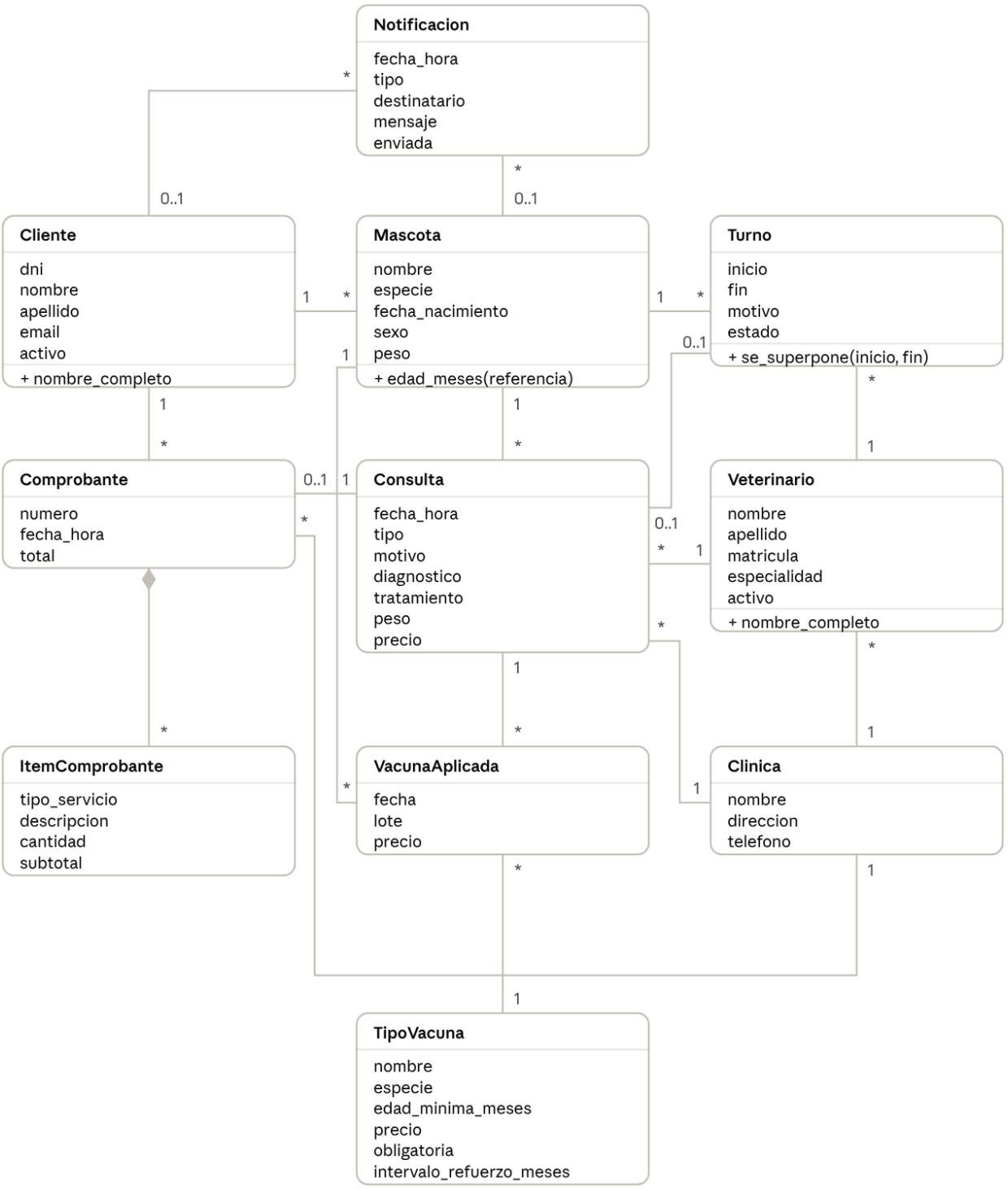


Diagrama de clases 1 · red, clientes, atención y facturación

Desde una muestra se reconstruye toda su cadena de custodia

Clases 2 de 4: derivación y custodia. Rombo = composición. Las cajas sombreadas están en el diagrama anterior.

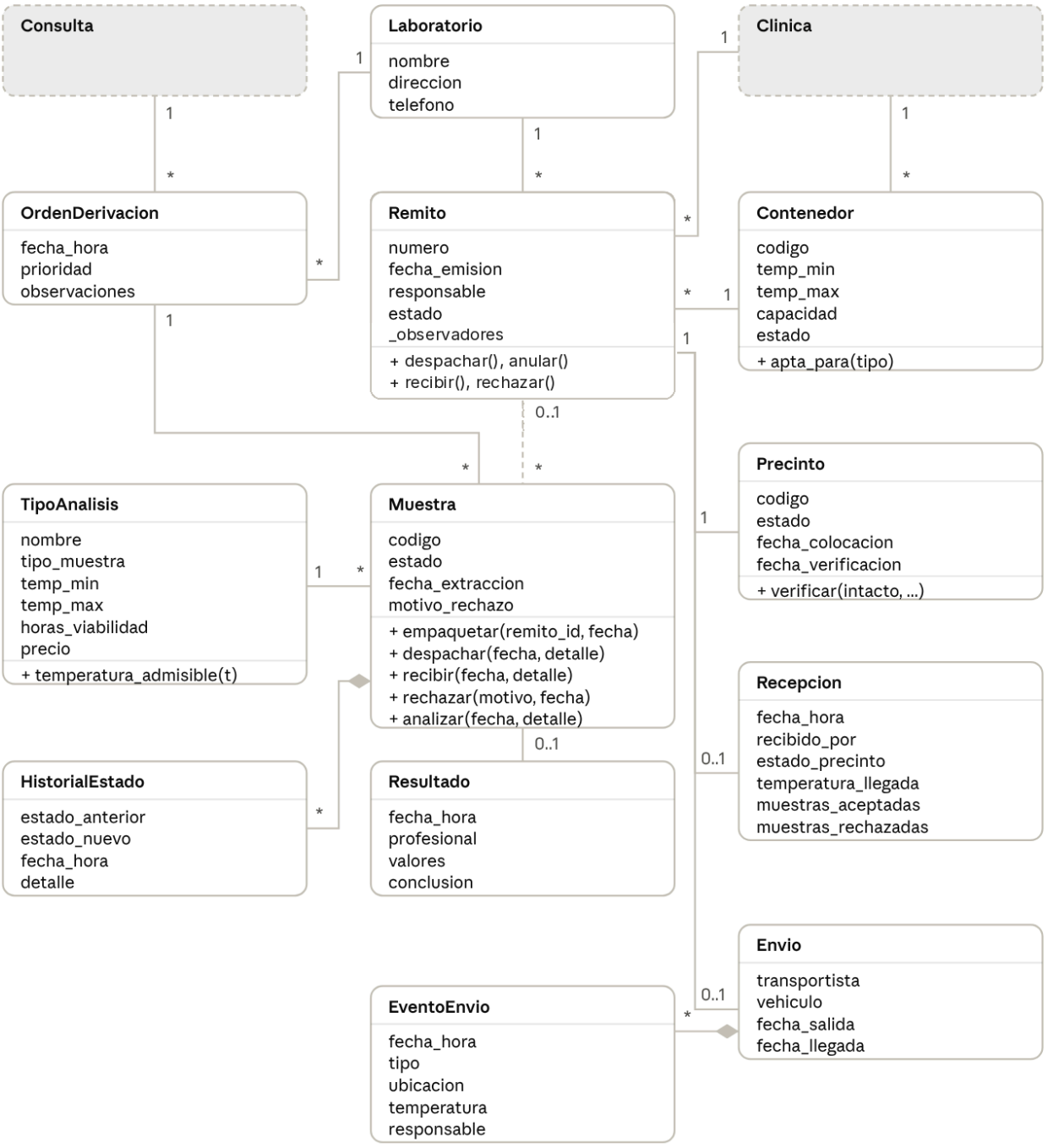


Diagrama de clases 2 · derivación y custodia

La muestra delega en su estado y avisa a sus observadores

Clases 3 de 4: patrones State y Observer de la muestra. Triángulo = herencia.

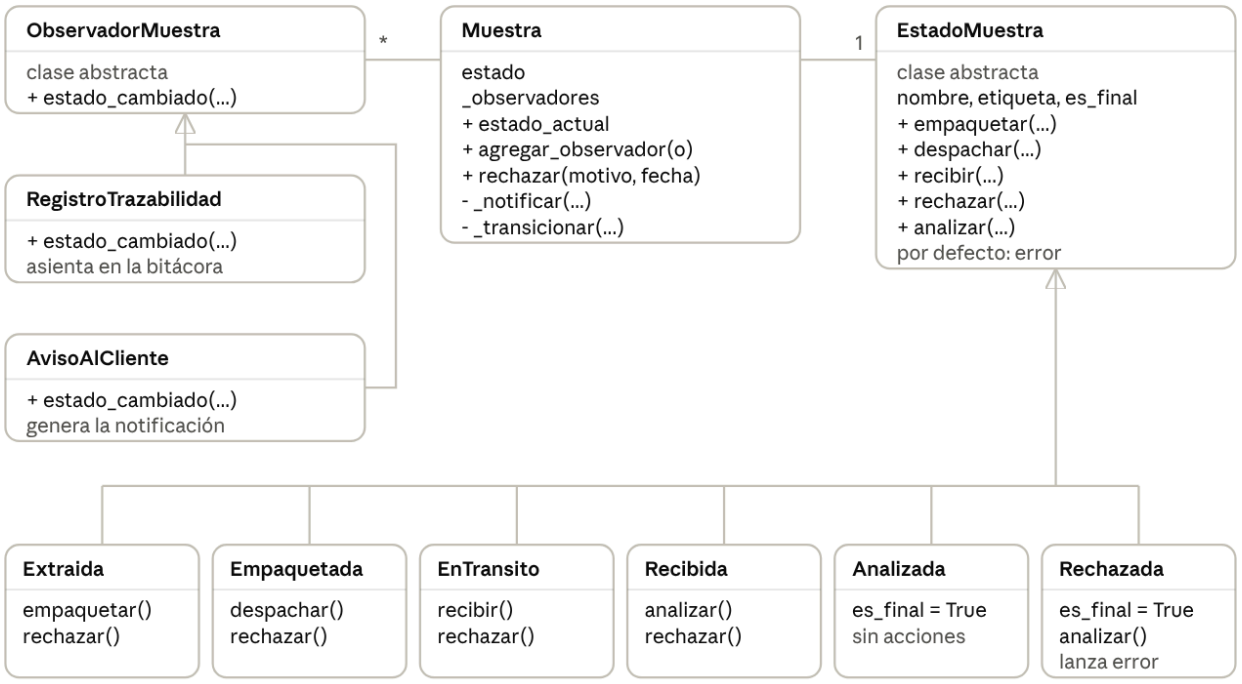


Diagrama de clases 3 · patrones State y Observer de la muestra

El remito también delega en su estado y avisa a su bitácora

Clases 4 de 4: patrones State y Observer del remito biológico. Triángulo = herencia.

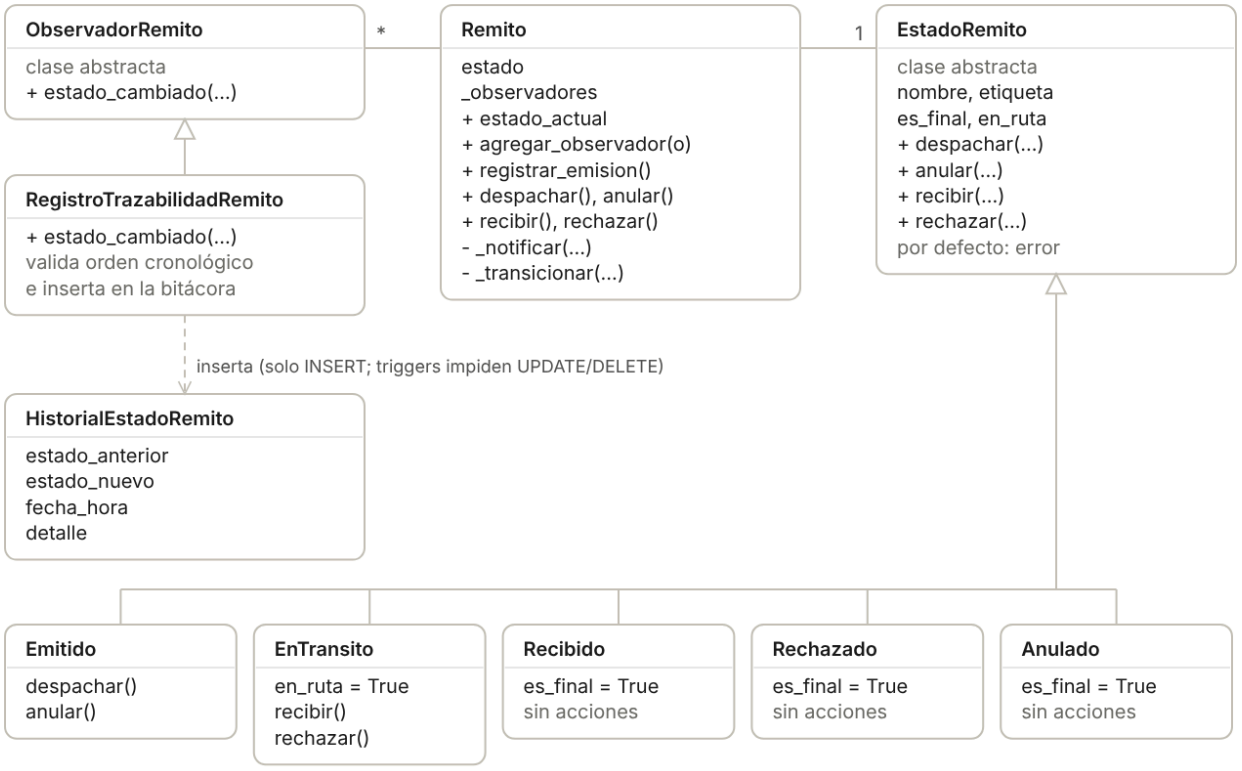
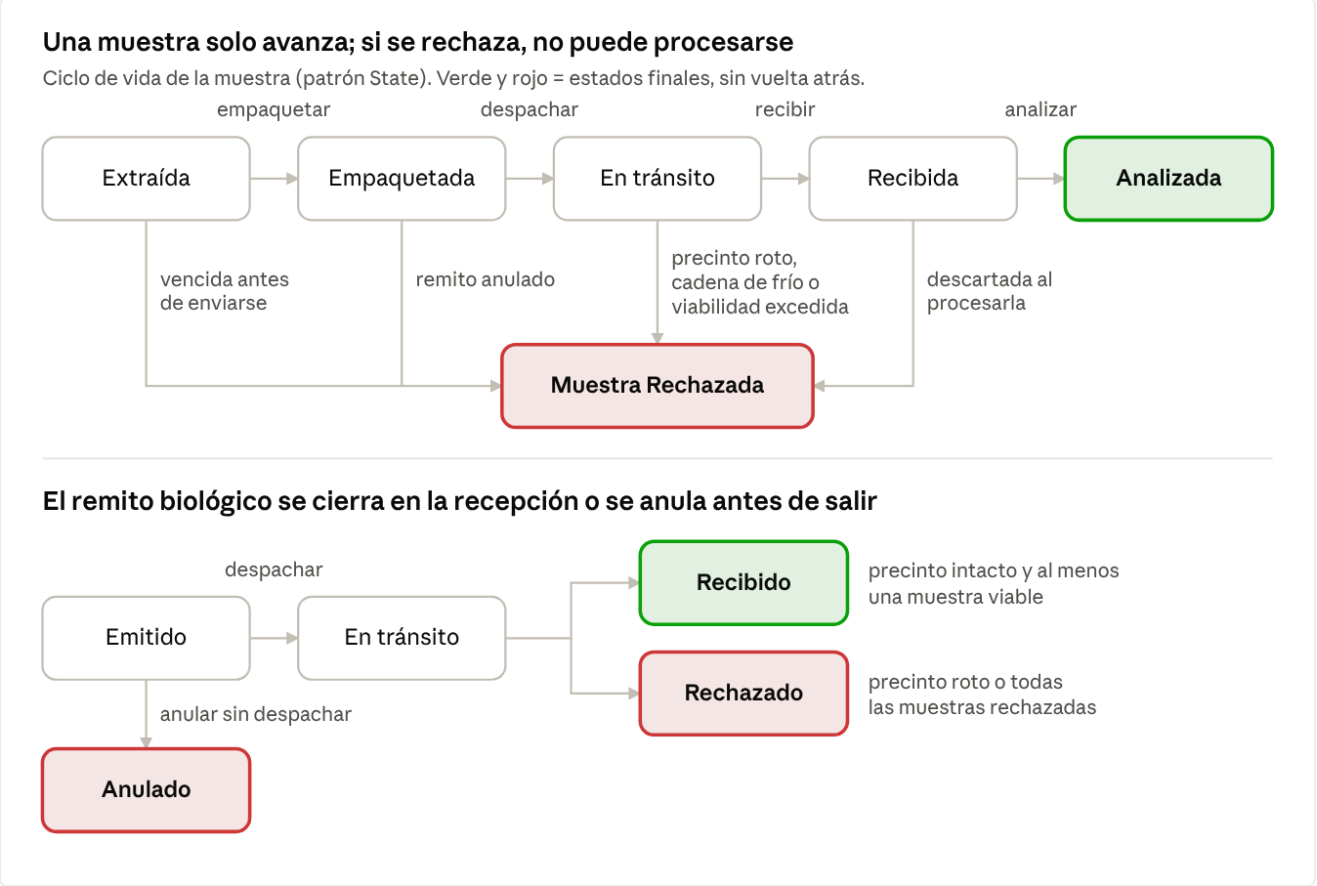


Diagrama de clases 4 · patrones State y Observer del remito biológico

4. Ciclo de vida de la muestra y del remito biológico

Una muestra recorre cinco estados y puede terminar en Muestra Rechazada desde la clínica, el transporte o el laboratorio. El remito recorre tres estados y se cierra como Recibido, Rechazado o Anulado. Ningún estado final admite cambios.



Estados de la muestra y del remito biológico

Cada flecha es un método de la clase de estado correspondiente (`Extraida.empaquetar()`, `Emitido.despachar()`, `EnTransito.recibir()`…); cualquier otra operación lanza `TransicionInvalidaError`. Cada transición se notifica a los observadores, que la asientan en la bitácora con fecha, estado anterior, estado nuevo y detalle (por ejemplo, el motivo del rechazo o quién anuló el remito). La bitácora valida además el orden cronológico.

En la recepción, cada muestra se evalúa en este orden y se rechaza ante la primera falla:

1. Precinto sanitario roto: se rechazan todas las muestras del remito y el remito pasa a Rechazado.
2. Alguna temperatura de la cadena de envío (salida, controles o llegada) fuera del rango del análisis.
3. Horas desde la extracción mayores que la viabilidad del análisis.

Si el precinto llegó intacto y al menos una muestra es viable, el remito pasa a Recibido; si todas se rechazaron, a Rechazado. En ambos casos la conservadora vuelve a quedar Disponible.

Un remito sin despachar puede anularse (`Emitido.anular()`): al abrir la conservadora el precinto queda roto, sus muestras pasan a Muestra Rechazada y la conservadora vuelve a Disponible. Un remito ya despachado no puede anularse: `EnTransito` no define `anular()`. El laboratorio también

puede rechazar una muestra Recibida, por ejemplo si llegó hemolizada. Una atención con comprobante emitido ya no admite órdenes, porque esos análisis no podrían facturarse.

> **Decisión de diseño:** la primera versión controlaba el remito con una tabla de transiciones. Se pasó a State por consistencia con la muestra y porque el remito sí cambia de comportamiento según su estado: solo un remito Emitido se despacha o anula, y solo uno En tránsito admite puntos de control y recepción (`en_ruta = True`).

5. Procesos de negocio

Los 7 procesos obligatorios están implementados, cada uno con su pantalla y su servicio; además se cubre en parte uno de los opcionales.

#	Proceso	Pantalla	Servicio / método	Qué hace
1	Administrar clientes	Clientes	`ClienteService`	Alta, modificación, baja lógica y búsqueda
2	Mascotas e historias clínicas	Mascotas	`MascotaService.historia_clinica`	ABM; la historia reúne atenciones, vacunas, análisis y esquema de vacunación. La baja cancela los turnos pendientes
3	Consultas y órdenes de derivación	Atención clínica	`ConsultaService.registrar`, `emitir_orden`	Registra la atención y las vacunas; la orden genera una muestra Extraída por análisis
4	Empaquetado	Empaquetado	`LogisticaService.empaquetar`	Coloca muestras en una conservadora y emite remito biológico y precinto sanitario
5	Cadena de envío	Remitos y envíos	`despachar`, `registrar_punto_control`	Registra salida y controles en ruta con temperatura y responsable
6	Recepción en laboratorio	Remitos y envíos (detalle)	`LogisticaService.recibir`	Verifica precinto, temperaturas y viabilidad; acepta o rechaza cada muestra y cierra el remito
7	Resultados y comprobantes	Resultados, Comprobantes	`LaboratorioService`, `ComprobanteService`	Carga resultados; emite el comprobante con consulta, vacunas y análisis realizados

Opcional y funciones adicionales

- Adicional: agenda de turnos sin superposición, que sostiene la validación obligatoria (pestaña Agenda).
- Opcional parcial: recordatorios de vacunas y avisos de resultados listos o muestras rechazadas (pestaña Avisos).
- Adicional: control de disponibilidad de conservadoras: quedan En uso mientras viajan y vuelven a Disponible al recibirse.

6. Validaciones de negocio

Las cinco validaciones de la consigna se aplican en dos niveles: en el dominio/servicio (mensaje claro al usuario) y, cuando es posible, también en la base de datos como última defensa.

Validación de la consigna	Dónde se aplica	Cómo
Toda mascota tiene un responsable	`Mascota.__post_init__`, `MascotaService`, columna `cliente_id NOT NULL`	Sin cliente activo no se crea la mascota; no se puede dar de baja un cliente con mascotas activas
Precinto roto impide el procesamiento	`LogisticaService.recibir`, estado `Rechazada`	Todas las muestras del remito pasan a Muestra Rechazada; `Rechazada.analizar()` lanza error
Trazabilidad inmutable de estados	Observers `RegistroTrazabilidad` y `RegistroTrazabilidadRemito` + triggers SQL	Cada cambio de la muestra y del remito se inserta en su bitácora; los triggers abortan cualquier UPDATE o DELETE sobre ellas y cualquier cambio de un estado final
Sin superposición de turnos	`TurnoService.reservar`	Rechaza un turno que se solape con otro del mismo veterinario o de la misma mascota, y fuera de 8 a 20 h
Esquema de vacunas obligatorias	`VacunaService` al registrar la atención	Valida especie, edad mínima y refuerzo prematuro; alerta las obligatorias pendientes o vencidas

Validaciones adicionales del dominio

- La conservadora debe garantizar el rango térmico del análisis y no superar su capacidad; no se aceptan muestras repetidas en un mismo empaquetado.
- La recepción rechaza muestras si alguna temperatura de la cadena salió de rango (ruptura de cadena de frío) o si se superó su ventana de viabilidad.
- **Fechas coherentes:** ningún hecho (atención, orden, empaquetado, despacho, control, recepción, resultado, comprobante) puede fecharse en el futuro; la atención no puede ser anterior al nacimiento de la mascota ni el comprobante anterior a la atención; los eventos de la trazabilidad deben ser cronológicos.
- Una muestra en estado final (Analizada o Rechazada) y un remito cerrado (Recibido, Rechazado o Anulado) no pueden cambiar; lo garantizan también triggers.
- El comprobante solo se emite cuando no quedan muestras sin resultado, y no factura las rechazadas.
- DNI único de 7 u 8 dígitos y email con formato válido.

7. Patrones de diseño

Se aplicaron cinco patrones (la consigna exige dos); State y Observer sostienen directamente el núcleo de trazabilidad y se usan tanto en la muestra como en el remito.

Patrón	Archivo	Problema que resuelve
State	`dominio/muestra.py`, `dominio/remito.py`	Las operaciones válidas dependen del estado de la muestra y del remito
Observer	`dominio/muestra.py`, `dominio/remito.py`, `servicios/observadores.py`	Varios interesados reaccionan a cada cambio de estado
Template Method	`reportes/base.py`	Todos los reportes siguen el mismo algoritmo con pasos distintos
Singleton	`persistencia/conexion.py`	Una única conexión compartida para que las transacciones abarquen varios servicios
Repository	`persistencia/repositorios.py`	Separar el SQL de alta, baja y modificación de la lógica de negocio

State

Cada estado de la muestra (`Extraida`, `Empaquetada`, `EnTransito`, `Recibida`, `Analizada`, `Rechazada`) es una subclase de `EstadoMuestra`, y cada estado del remito (`Emitido`, `EnTransito`, `Recibido`, `Rechazado`, `Anulado`) una subclase de `EstadoRemito`. Cada una redefine solo las operaciones que admite; la entidad delega en su estado actual: `muestra.analizar()` llama a `estado_actual.analizar(muestra)`. Una operación no permitida lanza `TransicionInvalidaError`. Así se eliminan los `if estado == ...` dispersos y la regla "no se procesa una muestra rechazada" vive en un solo lugar: `Rechazada.analizar()`.

Observer

`Muestra` y `Remito` mantienen una lista de observadores y los notifican en cada transición. `RegistroTrazabilidad` y `RegistroTrazabilidadRemito` insertan el cambio en su bitácora (y verifican el orden cronológico); `AvisoAlCliente` genera una notificación cuando la muestra queda Analizada o Rechazada. Agregar otro efecto (por ejemplo, un mail real) no requiere tocar las entidades.

Template Method

`Reporte.generar()` fija el algoritmo: `validar_filtros → obtener_datos → procesar → resumir → datos_grafico`. Solo `obtener_datos` es abstracto; el resto son hooks con comportamiento por defecto que cada reporte redefine si lo necesita. La pantalla de reportes trata a los seis por igual.

Singleton

`BaseDatos.__new__` devuelve siempre la misma instancia. Su método `transaccion()` es anidable: solo la transacción más externa hace COMMIT o ROLLBACK, de modo que si falla cualquier paso de un empaquetado (remito, bitácora, precinto, muestras, contenedor) no queda nada a medias.

Repository

Un `Repositorio` genérico mapea cada dataclass con su tabla (`obtener`, `listar`, `insertar`, `actualizar`). Los servicios trabajan con objetos de dominio y no escriben SQL de ABM; las consultas con joins para listados y reportes quedan en servicios y reportes.

Por qué no se aplicaron patrones en otros lugares

Un patrón se justifica cuando resuelve un problema concreto; aplicarlo donde no hay variación de comportamiento solo agrega clases. Estos son los lugares evaluados y descartados:

Lugar	Patrón candidato	Decisión y motivo
Estados del turno (Pendiente, Atendido, Cancelado) y de la conservadora (Disponible, En uso, Baja)	State	No se aplicó: tienen tres estados sin comportamiento propio, solo una etiqueta y una verificación. State conviene cuando el comportamiento cambia según el estado, como en la muestra y el remito.
Reglas de rechazo en la recepción (precinto, frío, viabilidad)	Strategy / Chain of Responsibility	No se aplicó: son tres reglas fijas, en un orden fijo y en un solo método (`_motivo_rechazo`). Si se agregaran reglas por tipo de análisis, Chain of Responsibility sería la opción correcta.
Precio de la consulta según el tipo	Strategy	No se aplicó: el precio es un dato, no un comportamiento; alcanza con un diccionario en `config.py`. Strategy tendría sentido con promociones o descuentos.
Recorrer colecciones (muestras de un remito, filas de un reporte)	Iterator	Lo provee el lenguaje: listas, `for` y generadores de Python ya implementan el protocolo iterador.
Creación de repositorios y estados	Factory	Se usa de forma implícita: `_repo()` crea las clases de repositorio y los diccionarios `ESTADOS` / `ESTADOS_REMITO` funcionan como registro de los estados. No hizo falta una jerarquía de fábricas.
Manejo de errores en las vistas	Decorator	`@validar` es un decorador de Python con la misma idea: agrega el manejo de errores de negocio sin modificar cada vista.

Costo asumido del Singleton: es estado global; las pruebas lo reinician con `BaseDatos.reiniciar()` y el servidor corre con un único hilo. Para una aplicación local de un solo usuario es aceptable.

Template Method: el gráfico de cada reporte

Cada reporte declara también qué gráfico lo acompaña con un objeto `Grafico` (en `reportes/base.py`): categoría, series en orden fijo, título, tipo y unidad. El tipo se elige según el dato: **barras** horizontales, apiladas si hay varias series, para comparar categorías (derivaciones por clínica, facturación por servicio, dosis pendientes por vacuna, rechazos por transportista y causa, horas por etapa) y **columnas** en orden cronológico para la evolución semanal de tratamientos. El hook `datos_grafico` arma las filas y `ResultadoReporte.grafico_preparado()` las ordena y limita a 15 categorías; la pantalla (Chart.js) y el PDF (reportlab) dibujan ese mismo resultado, y cada serie conserva siempre su color.

8. Reportes

Se implementaron seis reportes no triviales (la consigna pide cuatro), que cubren los cinco sugeridos. Agrupados en Clínicas, Mascotas, Análisis y Facturación, con resumen, gráfico y descarga CSV/PDF.

Reporte	Qué muestra	Por qué no es trivial
Atenciones clínicas y derivaciones	Atenciones por clínica y tipo, cuántas derivaron, % de derivación, muestras y órdenes urgentes	Agregación con subconsultas correlacionadas y porcentajes
Muestras rechazadas por fallas en el transporte	Detalle de cada rechazo con causa clasificada (precinto, cadena de frío, viabilidad) y tasa por transportista	Fecha de rechazo tomada de la bitácora; clasificación y tasas calculadas
Tratamientos aplicados	Atenciones con diagnóstico, tratamiento y vacunas; dosis aplicadas por vacuna	Concatena vacunas por atención y resume dosis
Vacunas obligatorias pendientes	Mascotas con vacunas nunca aplicadas o con refuerzo vencido y días de atraso	Se calcula en Python con la lógica del esquema (edad, intervalos)
Facturación por clínica y servicio	Importes de consultas, vacunas y laboratorio por clínica, % del total y ticket promedio	Pivote de ítems por tipo de servicio
Tiempos de tránsito y procesamiento	Horas promedio de espera, tránsito, procesamiento y total por clínica y análisis	Pivotea la bitácora de estados con una CTE y calcula diferencias con julianday

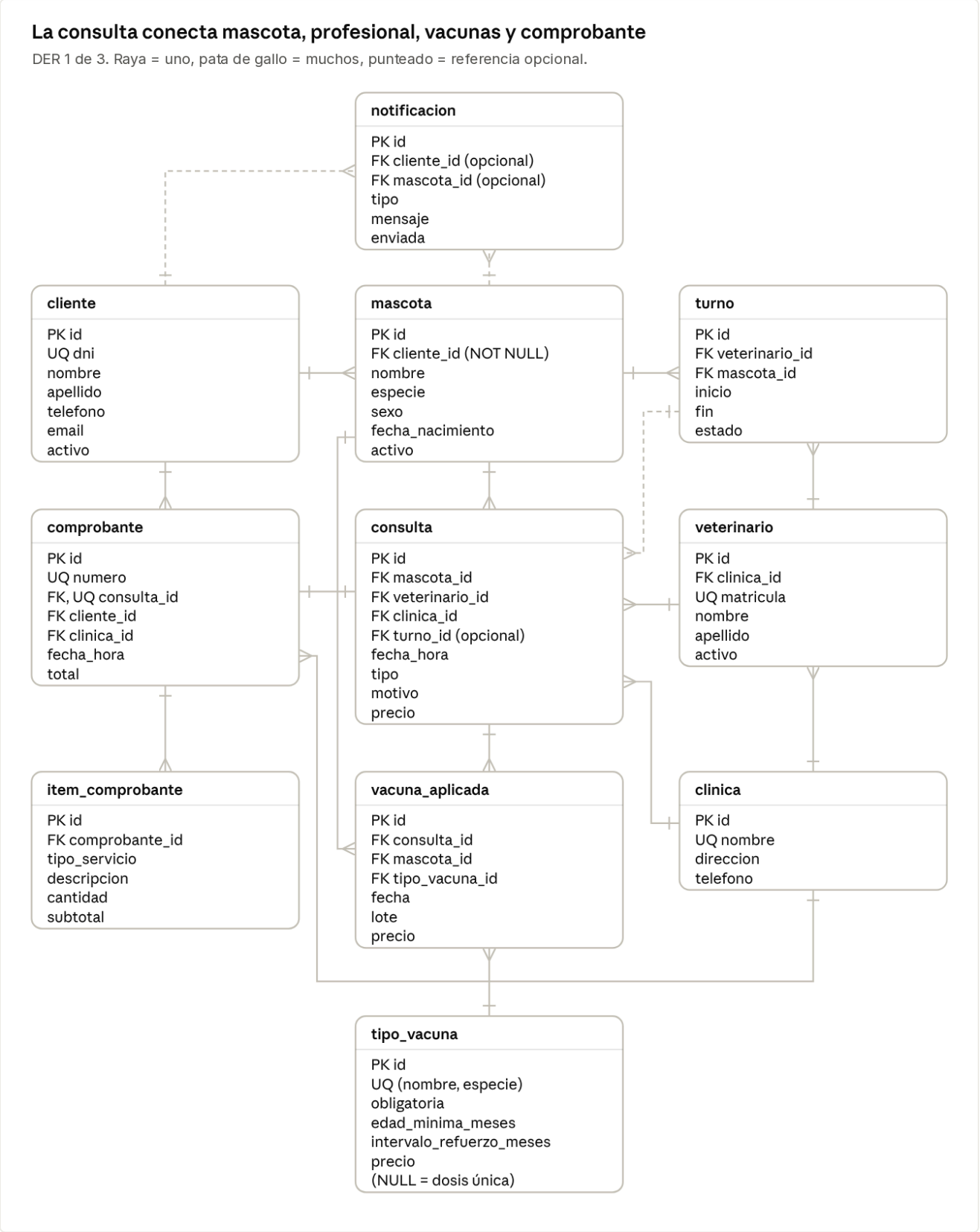
9. Arquitectura, persistencia, pruebas y ejecución

La aplicación se organiza en capas: la interfaz solo llama a servicios, los servicios usan el dominio y los repositorios, y nada fuera de persistencia abre conexiones.

Capa	Paquete	Responsabilidad
Presentación	`veterinaria/web`	Vistas Flask (una por área) y plantillas Bootstrap; el decorador `@validar` muestra los errores de negocio como alertas y vuelve a mostrar el formulario con lo cargado. Un manejador global evita errores 500 ante ids inexistentes
Servicios	`veterinaria/servicios`	Un servicio por proceso; transacciones y validaciones entre entidades
Dominio	`veterinaria/dominio`	Entidades con invariantes y ciclos de vida de la muestra y del remito
Reportes	`veterinaria/reportes`	Plantilla base y seis reportes
Persistencia	`veterinaria/persistencia`	Esquema SQL, conexión y repositorios

Persistencia. SQLite con 24 tablas, claves foráneas activas, restricciones CHECK sobre los estados y seis triggers: cuatro impiden modificar o borrar las bitácoras de la muestra y del remito, y dos impiden cambiar una muestra o un remito en estado final.

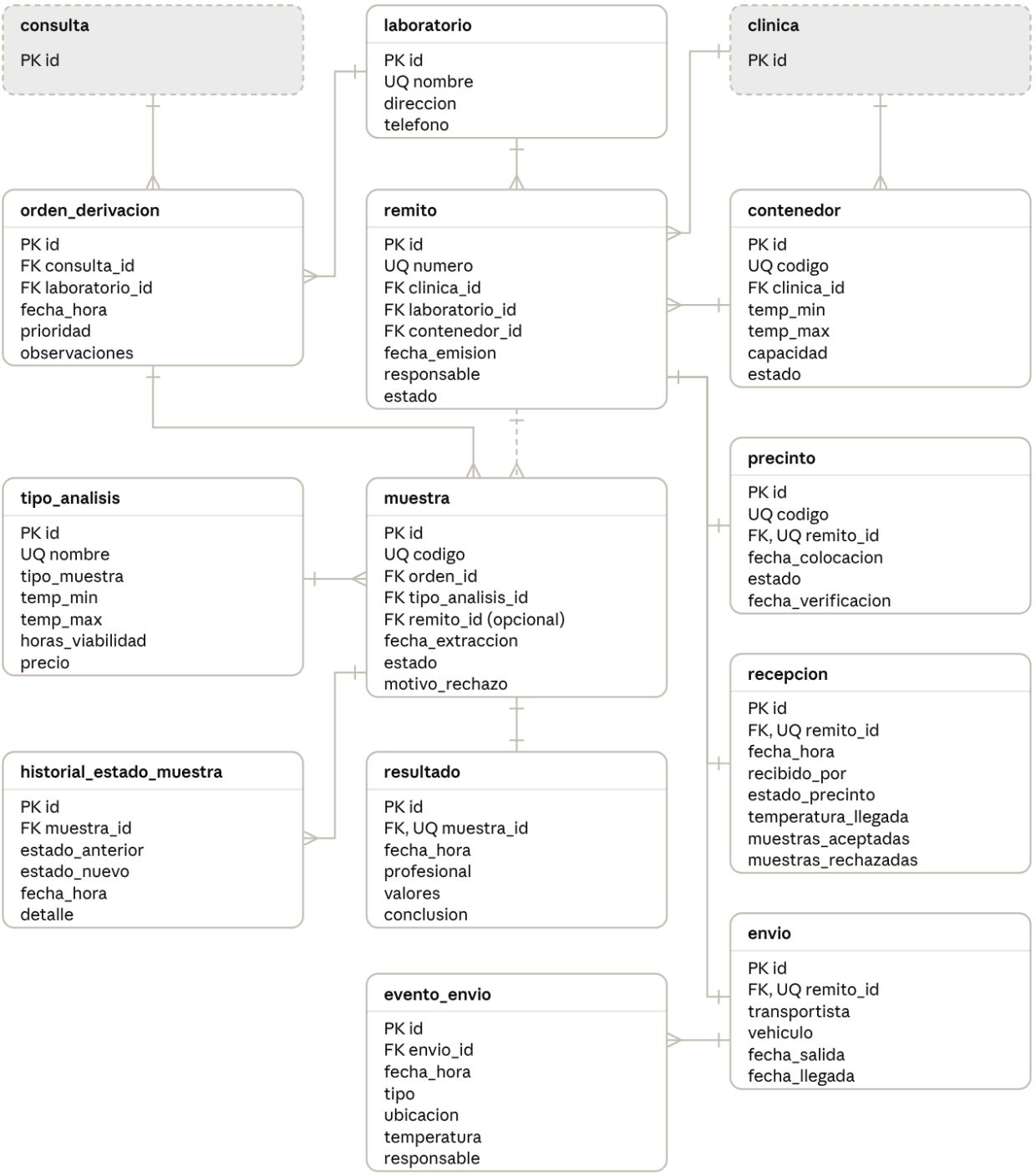
Diagrama entidad-relación. Muestra las tablas en tres partes (la tercera, la bitácora del remito), con sus claves y columnas principales; el esquema completo está en `esquema.sql`.



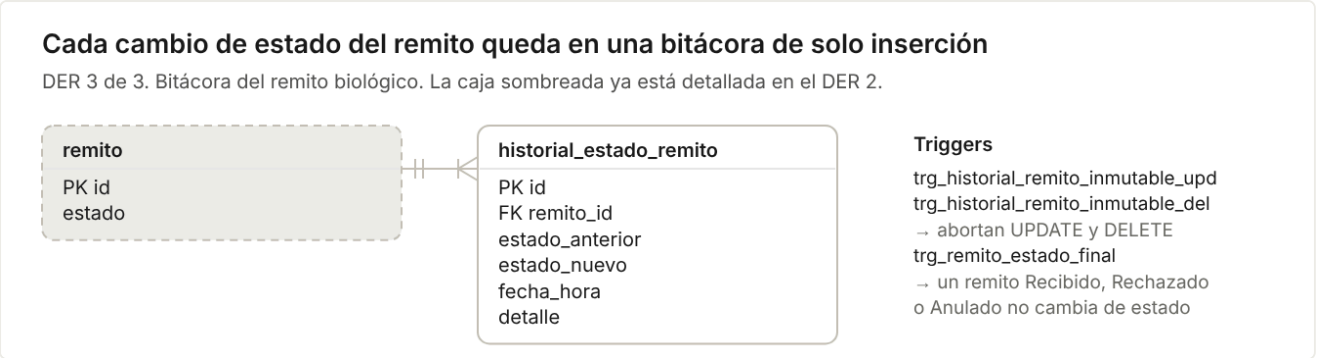
DER 1 · red, clientes, atención y facturación

La muestra se une al remito, y el remito reúne precinto, envío y recepción

DER 2 de 3. Las cajas sombreadas ya están detalladas en el DER 1.



DER 2 · derivación y cadena de custodia



DER 3 · bitácora del remito biológico

Datos de prueba. `datos_prueba.py` simula 75 días de actividad usando los mismos servicios, por lo que respeta todas las validaciones: 15 clientes, 25 mascotas, unas 280 atenciones y 280 muestras. Incluye precintos rotos, rupturas de cadena de frío, envíos demorados y mascotas con vacunas atrasadas, y deja para la demostración muestras sin empaquetar, un remito Emitido listo para despachar, remitos en camino y uno Anulado.

Pruebas. 43 pruebas automatizadas. `tests/test_negocio.py` (34): responsable obligatorio, DNI duplicado, superposición de turnos, horario, esquema de vacunas, recorrido completo con trazabilidad, rechazo por precinto roto, ruptura de cadena de frío, inmutabilidad de ambas bitácoras, transiciones inválidas de muestra y remito, conservadora no apta, comprobante con muestras pendientes, los seis reportes y sus gráficos, rechazo en el laboratorio, anulación de remito, orden posterior al comprobante, fechas futuras o incoherentes, muestras repetidas y baja de mascota con turnos. `tests/test_web.py` (9): todas las páginas, la descarga de los reportes en CSV y PDF, el circuito completo desde los formularios, ids inexistentes, la bitácora del remito en pantalla y la búsqueda por cliente en trazabilidad.

Ejecución (Python 3.11 o superior; dependencias: Flask y reportlab, esta última para exportar los reportes a PDF). En Windows, si `python` o `pip` no se reconocen, usar `py`:

```
py -m pip install -r requirements.txt   # solo la primera vez (instala Flask y reportlab)
py main.py                              # levanta el servidor y abre http://127.0.0.1:5000
py main.py --datos-prueba               # regenera la base de demostración
py -m unittest -v                       # corre las 43 pruebas
```

Guion sugerido para la demostración

1. Inicio: mostrar los indicadores y el circuito de muestras.
2. Atención clínica: elegir un paciente (aparece la alerta de vacunas) y usar “Registrar y derivar a laboratorio”.
3. Empaquetado: elegir la muestra nueva y una conservadora; se emiten remito y precinto.
4. Remitos y envíos: despachar y registrar un punto de control con 12 °C. Mostrar la bitácora de estados del remito.
5. Recepción (en el mismo remito): precinto intacto; la muestra se rechaza por cadena de frío y el remito queda Rechazado.
6. Resultados: intentar cargar el resultado de esa muestra (el sistema lo impide) y ver su trazabilidad.
7. Remitos anulados: abrir el remito Anulado de la demo y mostrar su bitácora.
8. Reportes: rechazos por transportista y tiempos de tránsito.